LESSON 06 · MISSION GUIDE

# The Full Moon

Errors, exceptions and debugging: scripts that survive real data.

| | |
|---|---|
| From | **Orrin of the Clockworks** |
| To | **Jen, Keeper of the Guild Ledger** |
| By | **Raven (third one tonight)** |

Keeper. The moon is full, so the Grimoire has failed, and your scroll script has joined it: row 5 of this moon's copy of the ledger says "n/a" where a reward should be. Quartermistress Pell sent me the traceback with nothing but "???". Three question marks. That's a lot for Pell.

The traceback is the most useful text Python will ever give you: what went wrong, where, in what order. Tonight you learn to read it, to catch the errors you expect, and to make the ones you don't expect loud instead of silent. Dalvric's approach was `On Error Resume Next`, which is how a difficulty-7 troll hunt went to two Bards and a lute. — Orrin

## What you'll be able to do after today

- [ ] Read a traceback bottom-up: the error type and message, then the line that caused it
- [ ] Tell the common exception types apart: `ValueError`, `TypeError`, `KeyError`, `IndexError`, `ZeroDivisionError`, `FileNotFoundError`
- [ ] Catch an expected error with `try` / `except` and carry on, without hiding the ones you didn't expect
- [ ] Use `else` and `finally` to keep the risky line alone in the `try`
- [ ] Refuse bad input on purpose with `raise` and a message a human can read
- [ ] Find a logic bug with a well-placed `print`, or with a breakpoint in VS Code

## Four big ideas

| | |
|---|---|
| **1. Errors are objects with types** | A `ValueError` means "right type, wrong content" (`float("n/a")`). A `TypeError` means "wrong type altogether" (`float(None)`). Each type can be caught on its own. |
| **2. Catch what you expect, let the rest crash** | `except ValueError:` is a plan. A bare `except:` is a blindfold, the Python version of `On Error Resume Next`. |
| **3. `try` / `except` / `else` / `finally`** | Risky line in `try`, what to do if it fails in `except`, what to do if it worked in `else`, what to do no matter what in `finally`. |
| **4. `raise` is for you** | When your own code gets input that makes no sense, stop with a clear message now, rather than produce a wrong number later. |

### Why this matters at work

- **Real data is broken.** A script that crashes on row 5 helps nobody; one that skips row 5 *and says so* is a colleague.
- **Silent errors are the expensive ones.** A crash costs five minutes. A wrong number before the Privy Council costs a week.
- **Debugging is a skill, not luck.** Reproduce, narrow down, look at the actual values. Same method every time.

### Three habits that bite

- Ten lines in one `try`: you won't know which one failed. Keep the risky line alone.
- `except Exception: pass`. Congratulations, the bug is now invisible.
- Fixing the symptom. `float("n/a")` fails because the data is wrong. Skip it *and log it*.

## How today works

15 MIN
**Lecture**
Lars crashes scripts on purpose and reads the wreckage

60 MIN
**Exercise sheet**
Seven ravens, from reading a traceback to a logged safety net

15 MIN
**Grimoire, page 6**
Dalvric's "robust" spell. Robust like a paper shield

## The ravens

| # | FROM | RAVEN | YOU'LL USE |
|---|---|---|---|
| 1 | Orrin | Read the traceback | `IndexError` `NameError`, line numbers |
| 2 | Pell | Catch it | `try / except ValueError / else` |
| 3 | Captain Siv | Which error is it? | several `except` blocks |
| 4 | Captain Siv | Raise your own | `raise ValueError(...)`, `finally`, `isinstance` |
| 5 | Pell | The broken file | `except (A, B) as e`, `type(e).__name__` |
| 6 | The Queen | Print debugging | `print()` in the loop, breakpoints |
| 7 ⭐ | Orrin | A safety net | a reusable `safe_float()` + a log file |

### You're done when...

- [ ] `my_work/` has `ex1` to `ex6` (and `ex7` if you're brave), each printing the expected output
- [ ] Dalvric's page prints the two expected lines, one commit per bug, and your commit message names the double-counted quest
- [ ] Everything is pushed to `lesson-06` and you've opened a pull request




**Next lesson: The Armoury.** You keep copying the same cleaning functions into every file. Time to forge them into a module of your own, borrow from Python's standard library, and install your first package with `pip`. NumPy and pandas are waiting on the other side.